

PRÁCTICA 11

Seguridad en Aplicaciones Web

Auditoría de OWASP Juice Shop

SACZN · Cybersecurity

OWASP Juice Shop · Gobuster · Nikto · Burp Suite · Kali Linux

0. Introducción y alcance

El presente informe documenta la auditoría de seguridad realizada sobre la aplicación web deliberadamente vulnerable OWASP Juice Shop, desplegada de forma local mediante contenedor Docker. El objetivo es identificar y documentar debilidades de seguridad a través de técnicas de reconocimiento, enumeración, análisis de cabeceras, análisis de aleatoriedad de tokens de sesión y explotación de vulnerabilidades, siguiendo una metodología equivalente a la empleada en auditorías web profesionales (alineada con OWASP Testing Guide y PTES).

0.1 Entorno de pruebas

Parámetro	Detalle
Sistema atacante	Kali Linux
Aplicación objetivo	OWASP Juice Shop (bkimminich/juice-shop)
Método de despliegue	Docker (docker run -d -p 3000:3000 bkimminich/juice-shop)
URL objetivo	http://localhost:3000 / http://127.0.0.1:3000
Herramientas empleadas	Nmap, Gobuster, Nikto, curl, Burp Suite Community Edition

0.2 Metodología

La auditoría se estructuró en las siguientes fases, siguiendo el orden habitual de un pentest de aplicación web: reconocimiento y mapeo de superficie, enumeración de contenido (fuzzing), revisión de ficheros de configuración pública, análisis de cabeceras de seguridad y cifrado, análisis de la aleatoriedad de tokens de sesión, y finalmente explotación de vulnerabilidades identificadas.

Nota metodológica: durante la auditoría el contenedor Docker se detuvo de forma inesperada (exit code 255) y fue necesario recrearlo. Este tipo de incidencia operativa es habitual en entornos de laboratorio y se documenta aquí por transparencia metodológica.

1. Fase inicial: herramientas empleadas y mapeo de superficie

La fase de reconocimiento inicial tiene como objetivo identificar los servicios expuestos por el objetivo y obtener una primera impresión de la tecnología subyacente. Se empleó Nmap con detección de versión y scripts por defecto contra el puerto identificado como expuesto por la aplicación.

1.1 Escaneo de puertos y servicios (Nmap)

```
$ sudo nmap -sV -sC -p 3000 localhost

Starting Nmap 7.99 at 2026-06-10 23:14 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000072s latency).

PORT      STATE SERVICE VERSION
3000/tcp   open  ppp?
| fingerprint-strings:
|   GetRequest:
|     HTTP/1.1 200 OK
|     Access-Control-Allow-Origin: *
|     X-Content-Type-Options: nosniff
|     X-Frame-Options: SAMEORIGIN
|     Feature-Policy: payment 'self'
|     X-Recruiting: /#/jobs
|     Cache-Control: public, max-age=0
|     Content-Type: text/html; charset=UTF-8
|     <title>OWASP Juice Shop</title>
|   HTTPOptions:
|     HTTP/1.1 204 No Content
|     Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
|   Help, NCP, RPCCheck:
|     HTTP/1.1 400 Bad Request

1 service unrecognized despite returning data.
Nmap done: 1 IP address (1 host up) scanned in 12.07 seconds
```

Salida abreviada para legibilidad (se omite el bloque de fingerprint en formato hexadecimal).

1.2 Análisis de los resultados

Nmap no logró identificar el servicio por nombre ("ppp?"), lo cual es esperable al tratarse de un servidor Node.js/Express sirviendo contenido sobre HTTP en un puerto no estándar (3000). Sin embargo, el fingerprinting de aplicación reveló información clave para las siguientes fases:

- **Ausencia de HTTPS:** el servidor responde sobre HTTP plano, sin TLS — toda la comunicación viaja sin cifrar.
- **Access-Control-Allow-Origin: ***: presente en la respuesta, indica CORS permisivo abierto a cualquier dominio.
- **X-Recruiting: /#/jobs**: cabecera no estándar que filtra información interna (enlace a sección de empleo).

- **X-Content-Type-Options y X-Frame-Options:** presentes, mitigan parcialmente clickjacking y MIME-sniffing.

Este mapeo inicial define la superficie de ataque: un único servicio HTTP en el puerto 3000, sin cifrado de transporte, sirviendo tanto el frontend (SPA Angular) como una API REST backend.

2. Evidencias del proceso de fuzzing de directorios y archivos

Se utilizó **Gobuster** en modo de enumeración de directorios (dir) con el diccionario común de dirb para descubrir rutas y archivos no enlazados directamente desde la interfaz de la aplicación.

2.1 Ajuste inicial necesario

La primera ejecución de **Gobuster** reveló que Juice Shop devuelve código 200 incluso para rutas inexistentes (comportamiento típico de SPAs). Fue necesario excluir las respuestas por longitud de contenido (9903 bytes) para obtener resultados útiles:

```
$ gobuster dir -u http://localhost:3000 -w /usr/share/wordlists/dirb/common.txt -t 50
[...] the server returns a status code that matches the provided options
for non existing urls. Please exclude the response length
```

2.2 Ejecución corregida y resultados

```
(kali㉿kali)-[~]
└─$ gobuster dir -u http://localhost:3000 -w /usr/share/wordlists/dirb/common.txt -t 50 --exclude-length 9903

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://localhost:3000
[+] Method: GET
[+] Threads: 50
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] Exclude Length: 9903
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

apis (Status: 500) [Size: 2992]
api (Status: 500) [Size: 2990]
assets (Status: 301) [Size: 156] [→ /assets/]
ftp (Status: 200) [Size: 11317]
media (Status: 301) [Size: 155] [→ /media/]
profile (Status: 500) [Size: 1043]
promotion (Status: 200) [Size: 5863]
redirect (Status: 500) [Size: 3113]
robots.txt (Status: 200) [Size: 28]
restricted (Status: 500) [Size: 3004]
restaurants (Status: 500) [Size: 3006]
restored (Status: 500) [Size: 3000]
rest (Status: 500) [Size: 2992]
restore (Status: 500) [Size: 2998]
Video (Status: 200) [Size: 10075518]
video (Status: 200) [Size: 10075518]
Progress: 4613 / 4613 (100.00%)

Finished
```

Figura 1. Resultado de Gobuster excluyendo respuestas de longitud 9903 bytes.

2.3 Análisis de hallazgos

Ruta	Estado	Hallazgo
/ftp	200 — Accesible	Directorio FTP expuesto públicamente, confirmado también en robots.txt
/api , /apis	500	Endpoints de API expuestos; el error revela información de la pila tecnológica
/rest	500	API REST backend accesible directamente
/profile	500	Ruta de perfil accesible sin autenticación previa
/redirect	500	Posible mecanismo de redirección — riesgo de Open Redirect
/restricted	500	Ruta restringida accesible
/video	200 — 10 MB	Archivo multimedia de gran tamaño expuesto públicamente
/assets , /media	301	Redirección a recursos estáticos — comportamiento esperado

El hallazgo más relevante es la accesibilidad del directorio /ftp, que coincide con la entrada Disallow: /ftp detectada en robots.txt. Esta ruta se confirma y amplía en la sección siguiente.

3. Revisión de robots.txt y sitemap.xml

Los ficheros robots.txt y sitemap.xml son frecuentemente pasados por alto en el desarrollo, pero pueden filtrar rutas internas o revelar configuraciones incompletas. Se consultaron directamente con curl.

3.1 robots.txt

```
(kali@kali)-[~]
$ curl http://localhost:3000/robots.txt
User-agent: *
Disallow: /ftp
```

Figura 2. Contenido de robots.txt obtenido mediante curl.

El fichero contiene una única entrada: Disallow: /ftp. Aunque la directiva robots.txt solo indica a los rastreadores que no indexen esa ruta, no impide el acceso directo — de hecho, confirma su existencia. Esto constituye una filtración de información ("security through obscurity" fallida): el propio fichero diseñado para SEO termina revelando un directorio sensible.

3.2 sitemap.xml

```
(kali@kali)-[~]
$ curl http://localhost:3000/sitemap.xml
<!--
~ Copyright (c) 2014-2026 Bjoern Kimminich & the OWASP Juice Shop contributors
.
~ SPDX-License-Identifier: MIT
-->

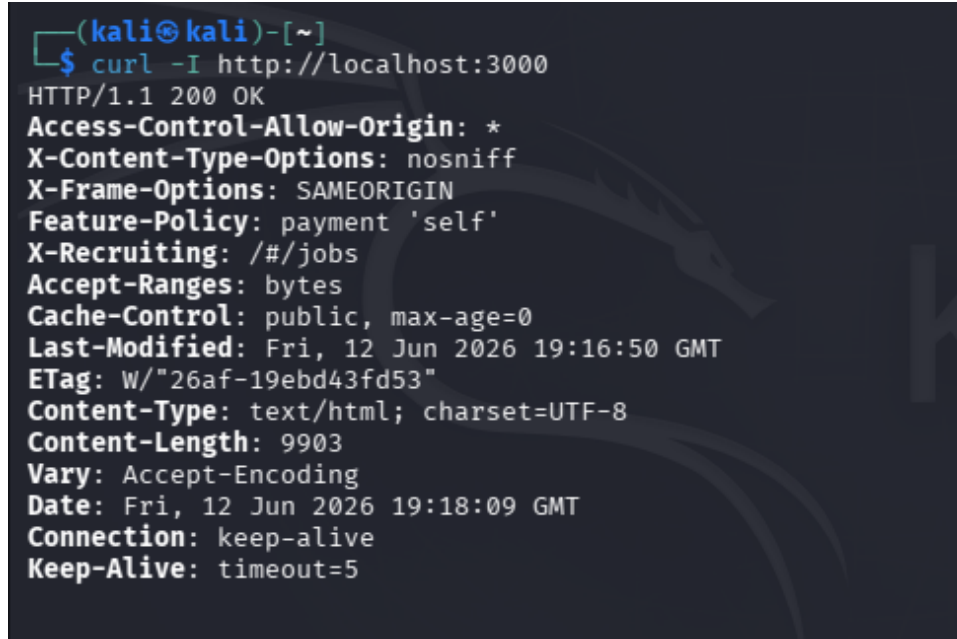
<!doctype html>
<html lang="en" data-beasties-container>
<head>
  <meta charset="utf-8">
  <title>OWASP Juice Shop</title>
  <meta name="description" content="Probably the most modern and sophisticated i
nsecure web application">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <style>@font-face{font-family:'VT323';font-style:normal;font-weight:400;font-d
isplay:swap;src:url(https://fonts.gstatic.com/s/vt323/v18/pxiKyp0ihIEF2isQFJXGdg
.woff2) format('woff2');unicode-range:U+0102-0103, U+0110-0111, U+0128-0129, U+0
168-0169, U+01A0-01A1, U+01AF-01B0, U+0300-0301, U+0303-0304, U+0308-0309, U+032
3, U+0329, U+1EA0-1EF9, U+20AB;}@font-face{font-family:'VT323';font-style:normal
;font-weight:400;font-display:swap;src:url(https://fonts.gstatic.com/s/vt323/v18
/pxiKyp0ihIEF2isRFJXGdg.woff2) format('woff2');unicode-range:U+0100-02BA, U+02BD
-02C5, U+02C7-02CC, U+02CE-02D7, U+02DD-02FF, U+0304, U+0308, U+0329, U+1D00-1DB
F, U+1E00-1E9F, U+1EF2-1EFF, U+2020, U+20A0-20AB, U+20AD-20C0, U+2113, U+2C60-2C
7F, U+A720-A7FF;}@font-face{font-family:'VT323';font-style:normal;font-weight:40
0;font-display:swap;src:url(https://fonts.gstatic.com/s/vt323/v18/pxiKyp0ihIEF2i
```

Figura 3. Respuesta al solicitar sitemap.xml: la aplicación devuelve el HTML de la SPA en lugar de un XML real.

La petición a /sitemap.xml no devuelve un documento XML válido, sino el mismo HTML base de la SPA (comportamiento de fallback típico sin rutas de servidor configuradas). No representa una vulnerabilidad de seguridad per se, pero sí una configuración incompleta a nivel de buenas prácticas web.

4. Análisis de la fortaleza criptográfica y de las cabeceras web

4.1 Cabeceras HTTP (curl)

A terminal window with a dark background and light blue text. The prompt is '(kali㉿kali)-[~]'. The command '\$ curl -I http://localhost:3000' has been entered. The output shows an HTTP 200 OK response with various headers: Access-Control-Allow-Origin: *, X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, Feature-Policy: payment 'self', X-Recruiting: /#/jobs, Accept-Ranges: bytes, Cache-Control: public, max-age=0, Last-Modified: Fri, 12 Jun 2026 19:16:50 GMT, ETag: W/"26af-19ebd43fd53", Content-Type: text/html; charset=UTF-8, Content-Length: 9903, Vary: Accept-Encoding, Date: Fri, 12 Jun 2026 19:18:09 GMT, Connection: keep-alive, and Keep-Alive: timeout=5.

```
(kali㉿kali)-[~]  
$ curl -I http://localhost:3000  
HTTP/1.1 200 OK  
Access-Control-Allow-Origin: *  
X-Content-Type-Options: nosniff  
X-Frame-Options: SAMEORIGIN  
Feature-Policy: payment 'self'  
X-Recruiting: /#/jobs  
Accept-Ranges: bytes  
Cache-Control: public, max-age=0  
Last-Modified: Fri, 12 Jun 2026 19:16:50 GMT  
ETag: W/"26af-19ebd43fd53"  
Content-Type: text/html; charset=UTF-8  
Content-Length: 9903  
Vary: Accept-Encoding  
Date: Fri, 12 Jun 2026 19:18:09 GMT  
Connection: keep-alive  
Keep-Alive: timeout=5
```

Figura 4. Cabeceras de respuesta obtenidas con curl -I.

4.2 Análisis automatizado (Nikto)

Nikto confirma y amplía los hallazgos anteriores, señalando explícitamente la ausencia de varias cabeceras de seguridad recomendadas:

```

(kali㉿kali)-[~]
$ nikto -h http://localhost:3000
- Nikto v2.6.0

+ Target IP:      127.0.0.1
+ Target Hostname: localhost
+ Target Port:    3000
+ Platform:      Unknown
+ Start Time:     2026-06-12 15:26:35 (GMT-4)

+ Server: No banner retrieved
+ ERROR: Failed to check for updates: 403
+ [999986] /: Retrieved access-control-allow-origin header: *.
+ [999100] /: Uncommon header(s) 'x-recruiting' found, with contents: /#/jobs.
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [999996] /robots.txt: contains 1 entry which should be manually viewed. See: https://developer.mozilla.org/en-US/docs/Glossary/Robots.txt
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ ERROR: *** Error limit (20) reached for host, giving up. Last error: error reading HTTP response.
***
+ ERROR: *** Consider using mitmproxy to avoid TLS fingerprinting. ***
- STATUS: Completed 965 requests (~13% complete, ~10.5 minutes left): currently in plugin 'Nikto Tests'
- STATUS: Running average: Not enough data.
+ Scan terminated: 20 errors and 7 items reported on the remote host
+ End Time:       2026-06-12 15:28:10 (GMT-4) (95 seconds)

+ 1 host(s) tested

```

Figura 5. Resultado del escaneo Nikto sobre el objetivo.

4.3 Tabla consolidada de cabeceras de seguridad

Cabecera	Estado	Riesgo asociado
Access-Control-Allow-Origin: *	Presente (inseguro)	CORS abierto a cualquier origen; facilita ataques cross-origen
X-Content-Type-Options: nosniff	Presente	Correcto — mitiga MIME-sniffing
X-Frame-Options: SAMEORIGIN	Presente	Correcto — mitiga clickjacking básico
X-Recruiting (no estándar)	Presente	Fuga de información interna innecesaria
Content-Security-Policy	Ausente	Alto — sin mitigación activa frente a XSS
Strict-Transport-Security	Ausente	Alto — no se fuerza el uso de HTTPS
Permissions-Policy	Ausente	Sin restricción sobre APIs sensibles del navegador
Referrer-Policy	Ausente	Posible fuga de URLs internas vía cabecera Referer

4.4 Fortaleza criptográfica

El análisis de cifrado en este entorno arroja un resultado claro: la aplicación se sirve íntegramente sobre HTTP, sin TLS/SSL configurado. En consecuencia:

- Todo el tráfico, incluidas credenciales de login y tokens JWT, viaja en texto claro entre cliente y servidor.
- No es posible evaluar la fortaleza de cifrado de un canal TLS porque dicho canal no existe en este despliegue.
- En un entorno productivo real, esta ausencia de TLS sería por sí sola una vulnerabilidad crítica (CWE-319: Cleartext Transmission of Sensitive Information).

Conclusión: el principal hallazgo de fortaleza criptográfica no es la debilidad de un algoritmo, sino la ausencia total de cifrado de transporte — el riesgo más alto posible en esta categoría.

5. Uso del Sequencer de Burp Suite

El módulo Sequencer de Burp Suite evalúa la calidad de la aleatoriedad (entropía) de tokens generados por la aplicación. Una baja entropía implicaría que los tokens son predecibles y susceptibles de ser adivinados por un atacante.

5.1 Configuración de la captura en vivo

Se interceptó con Burp Suite la petición POST a /rest/user/login, configurando Firefox como cliente proxy (127.0.0.1:8080), dado que el navegador integrado de Burp no lograba conectar con el objetivo. La petición se envió al Sequencer mediante "Send to Sequencer":

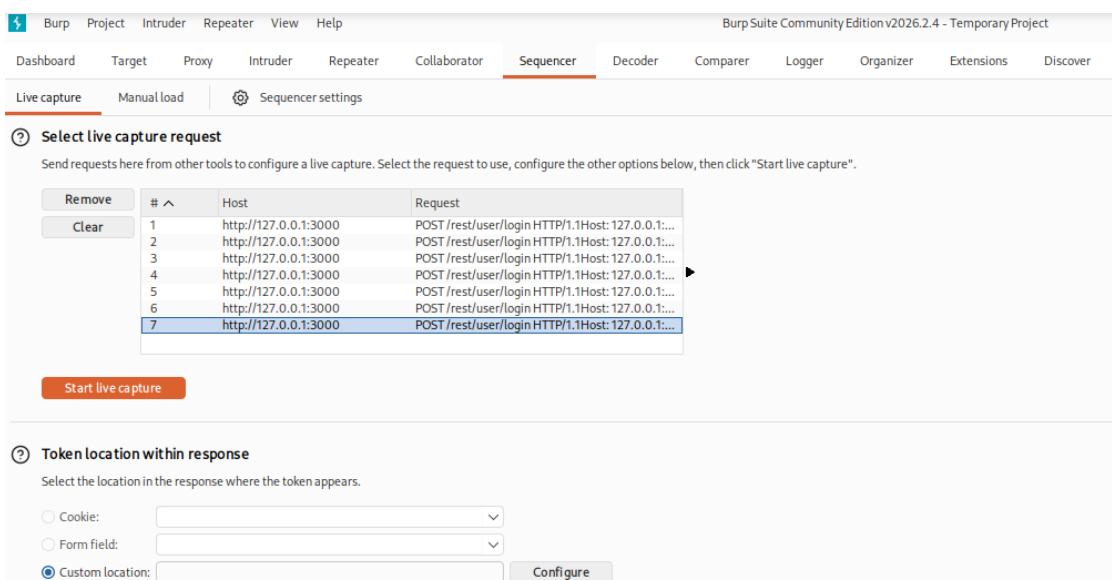


Figura 6. Configuración de la captura en vivo en Burp Sequencer, con 7 peticiones de muestra cargadas.

5.2 Localización del token

Dado que Juice Shop devuelve el token dentro de un objeto JSON (no como cookie), fue necesario definir manualmente su ubicación mediante "Custom location":

Parámetro	Valor configurado
Start after expression	"token":
End at delimiter	","bid"

Con esta configuración, Burp repitió automáticamente la petición de login 700 veces, capturando un token JWT distinto en cada respuesta para su análisis estadístico.

5.3 Resultado del análisis de entropía

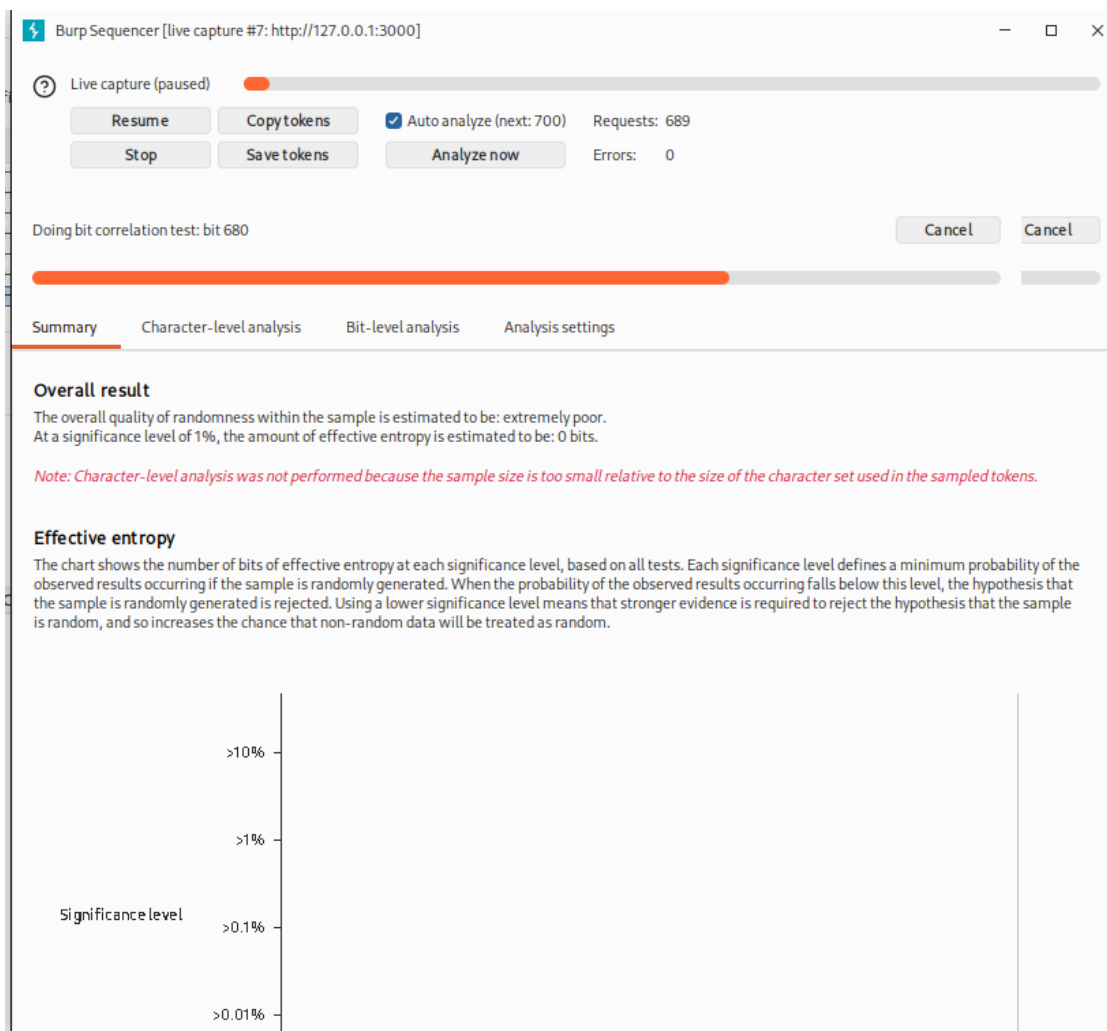


Figura 7. Resumen del análisis de Sequencer: calidad de aleatoriedad estimada como "extremely poor", 0 bits de entropía efectiva.

5.4 Interpretación crítica del resultado

El resultado —"extremely poor", 0 bits de entropía efectiva— podría interpretarse erróneamente como una vulnerabilidad crítica. Sin embargo, se trata de una limitación metodológica del análisis, no de un fallo real de la aplicación:

- El token analizado es un JWT completo (HEADER.PAYLOAD.SIGNATURE), donde el HEADER y PAYLOAD son prácticamente idénticos entre muestras al usar el mismo usuario.
- Como la porción fija representa la mayoría de los 732 caracteres del token, el test estadístico interpreta la ausencia de variación como entropía nula.
- La parte verdaderamente aleatoria del JWT es exclusivamente la firma criptográfica (tercer segmento), que no fue aislada en este análisis.

Recomendación metodológica: para medir entropía representativa en JWT, el Sequencer debe configurarse para analizar únicamente el tercer segmento (firma), aislado con un delimitador en el segundo punto del token. Este hallazgo se documenta como una observación técnica, no como una vulnerabilidad explotable.

6. Explicación de vulnerabilidades identificadas

Se documentan a continuación dos vulnerabilidades identificadas y explotadas durante la auditoría, ambas relacionadas con el mecanismo de autenticación de la aplicación.

6.1 Vulnerabilidad 1 — Credenciales de administrador débiles

Campo	Detalle
Clasificación	CWE-521: Weak Password Requirements / CWE-798: Use of Hard-coded Credentials
Severidad	Alta
Componente afectado	Endpoint POST /rest/user/login

Se logró iniciar sesión en la cuenta administradora (admin@juice-sh.op) utilizando la contraseña admin123, una credencial pública y ampliamente documentada. La petición capturada en Burp Suite confirma el envío de estas credenciales:

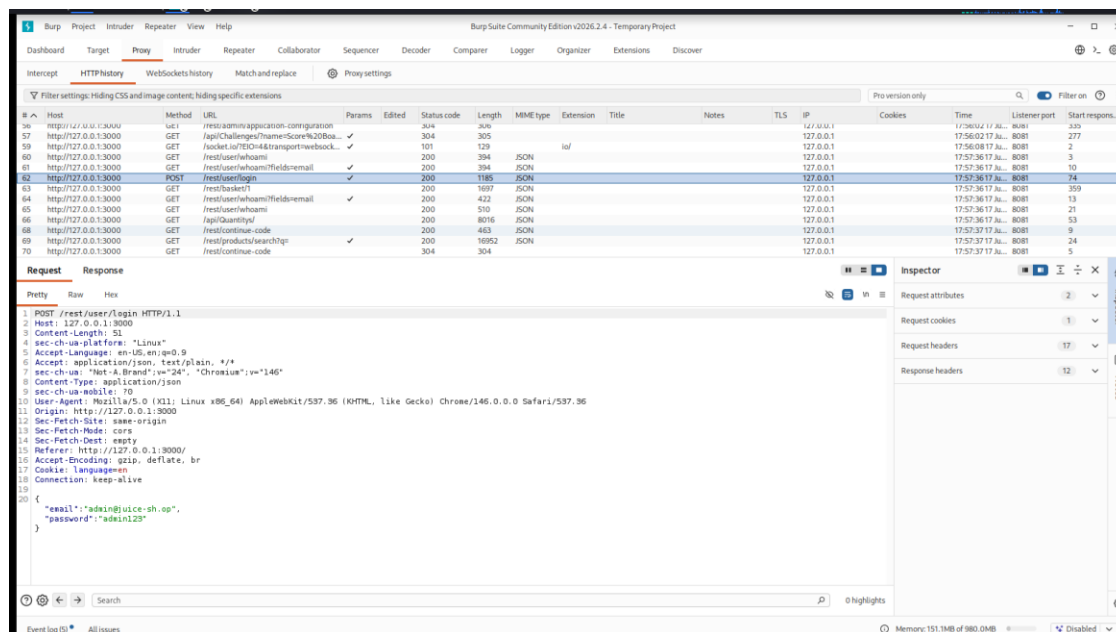


Figura 8. Petición POST /rest/user/login con credenciales admin@juice-sh.op / admin123 capturada en Burp Suite.

La propia aplicación confirmó la explotación exitosa, notificando la resolución de dos retos internos:

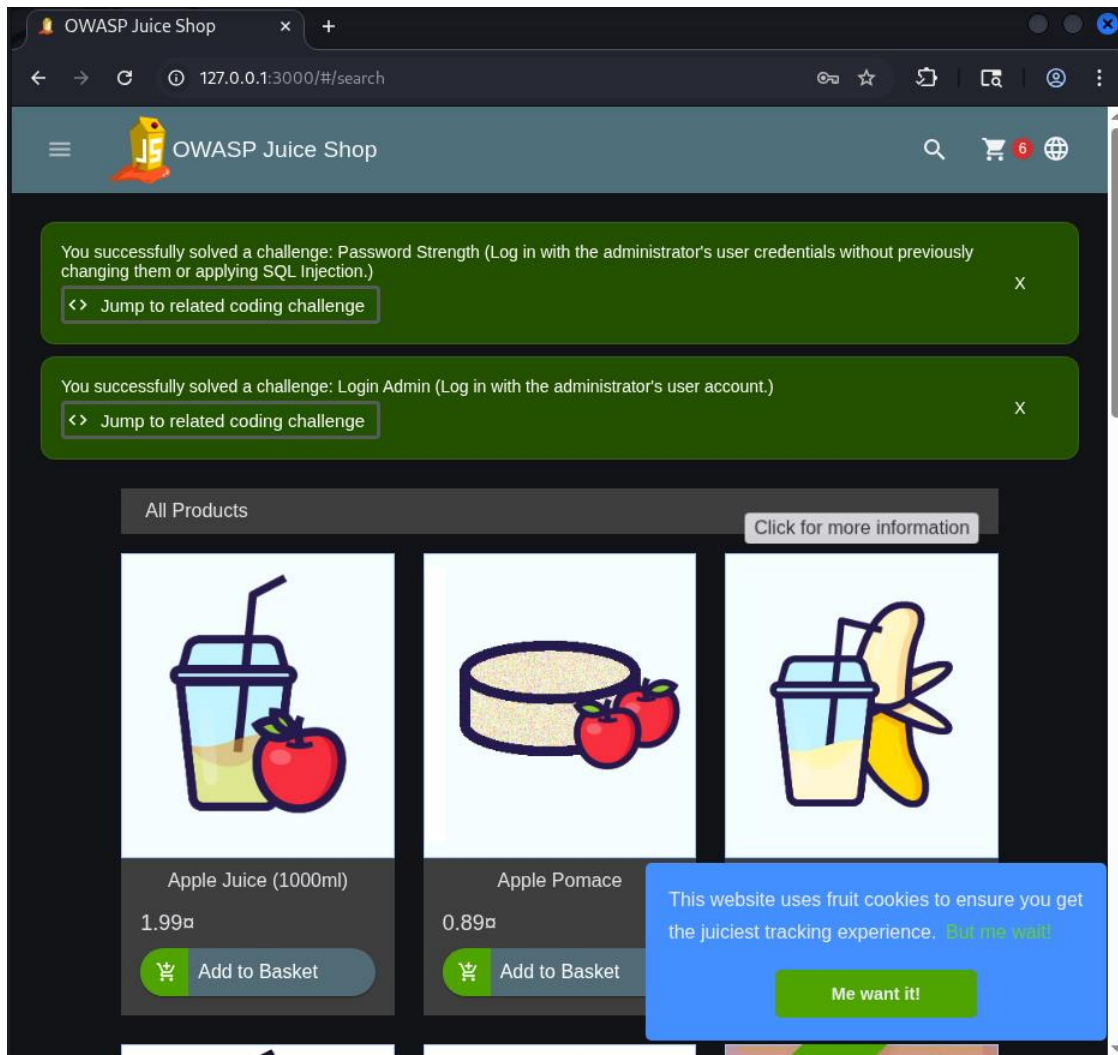


Figura 9. Confirmación: retos "Login Admin" y "Password Strength" resueltos tras el acceso con credenciales débiles.

Impacto: acceso completo a la cuenta administradora sin vulnerar ningún mecanismo técnico — solo aprovechando credenciales débiles conocidas públicamente.

Recomendación: forzar el cambio de contraseña en el primer inicio de sesión de cuentas privilegiadas, aplicar políticas de complejidad mínima y eliminar credenciales por defecto antes del despliegue en producción.

6.2 Vulnerabilidad 2 — Inyección SQL con bypass de autenticación

Campo	Detalle
Clasificación	CWE-89: SQL Injection / OWASP Top 10 A03:2021 — Injection
Severidad	Crítica
Componente afectado	Campo email del formulario de login — Endpoint POST /rest/user/login

El campo de email del formulario de login no sanea ni parametriza la entrada del usuario antes de construir la consulta SQL subyacente. Se introdujo el siguiente payload:

```
Email:      ' OR 1=1--  
Password: 123
```

La consulta resultante en el backend pasa de:

```
SELECT * FROM users WHERE email = '[input]' AND password = '[input]'
```

a algo equivalente a:

```
SELECT * FROM users WHERE email = '' OR 1=1--' AND password = '123'
```

La condición `OR 1=1` es siempre verdadera y `--` comenta el resto, neutralizando la verificación de contraseña. El servidor autentica al atacante como el primer usuario de la base de datos sin conocer ninguna credencial real.

Evidencia paso a paso

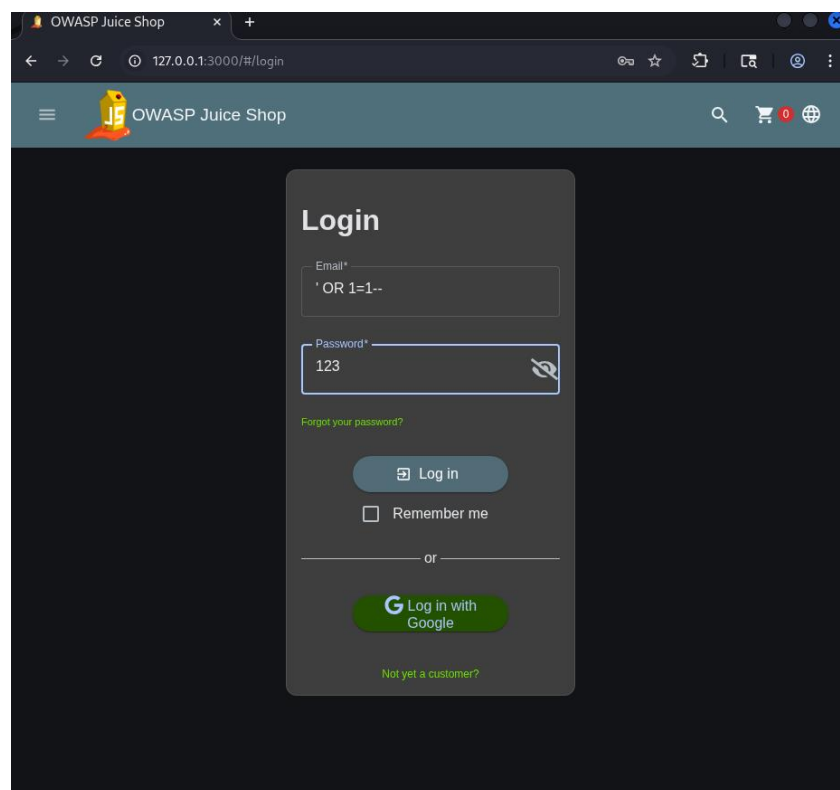


Figura 10. Payload de inyección SQL introducido en el formulario de login.

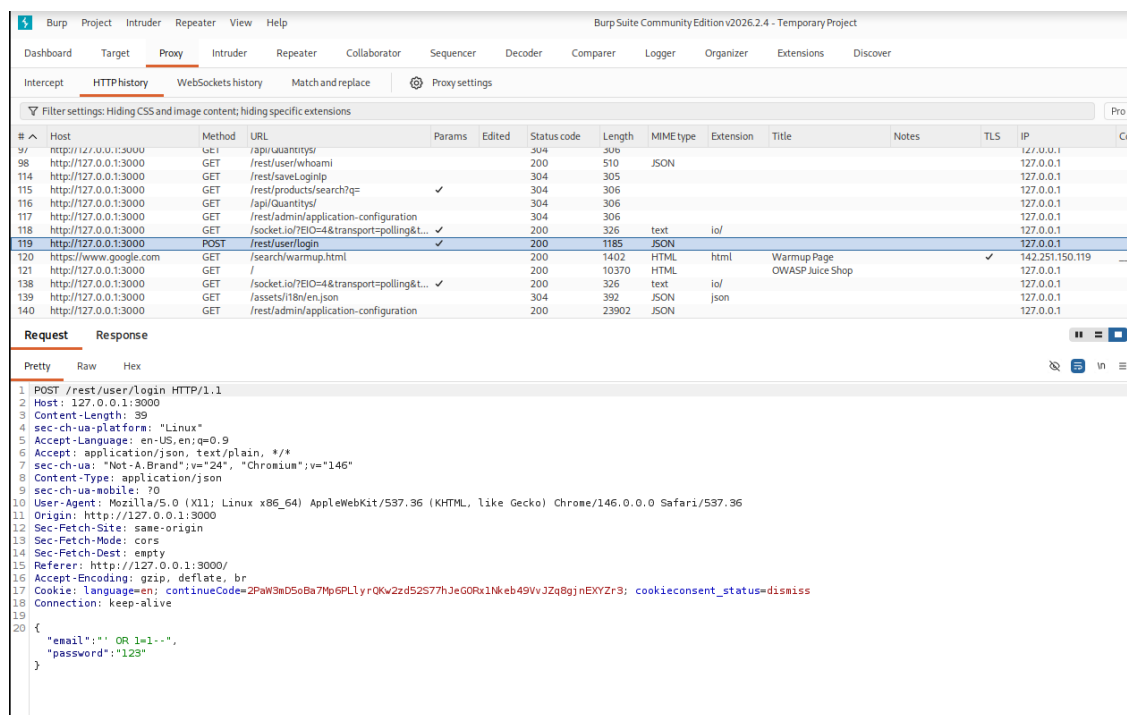


Figura 11. Petición HTTP capturada en Burp Suite con el payload ' OR 1=1-- en el campo email.

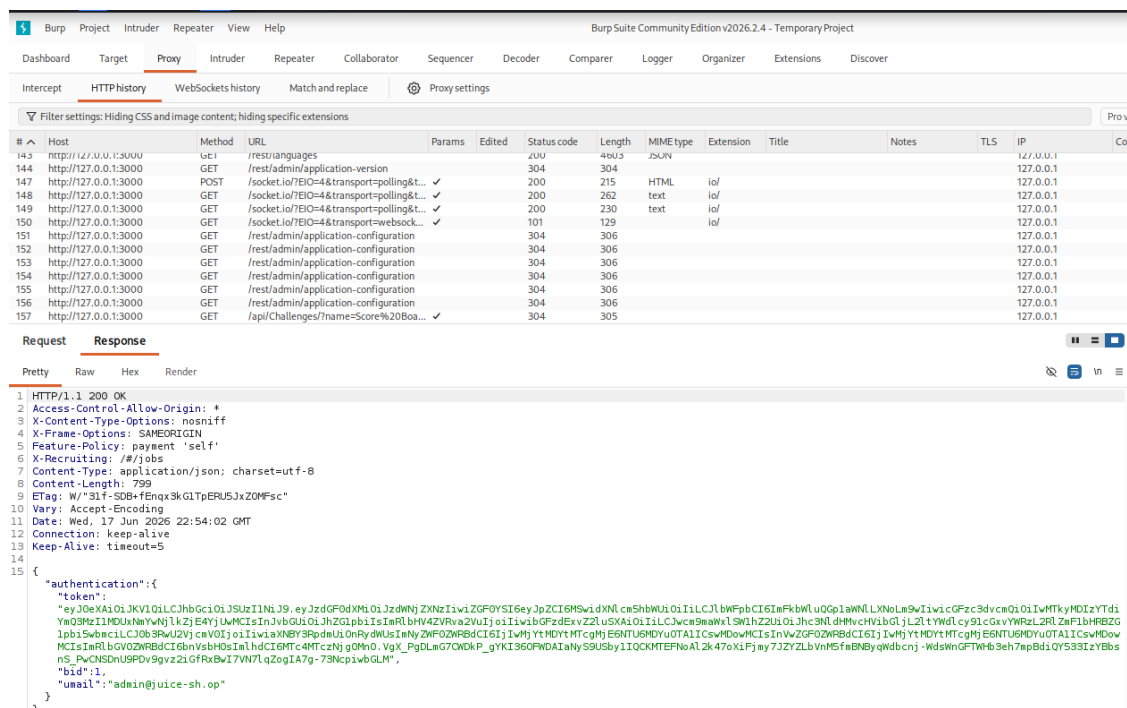


Figura 12. Respuesta del servidor: token JWT válido junto con "gmail":"admin@juice-sh.op", confirmando el bypass.

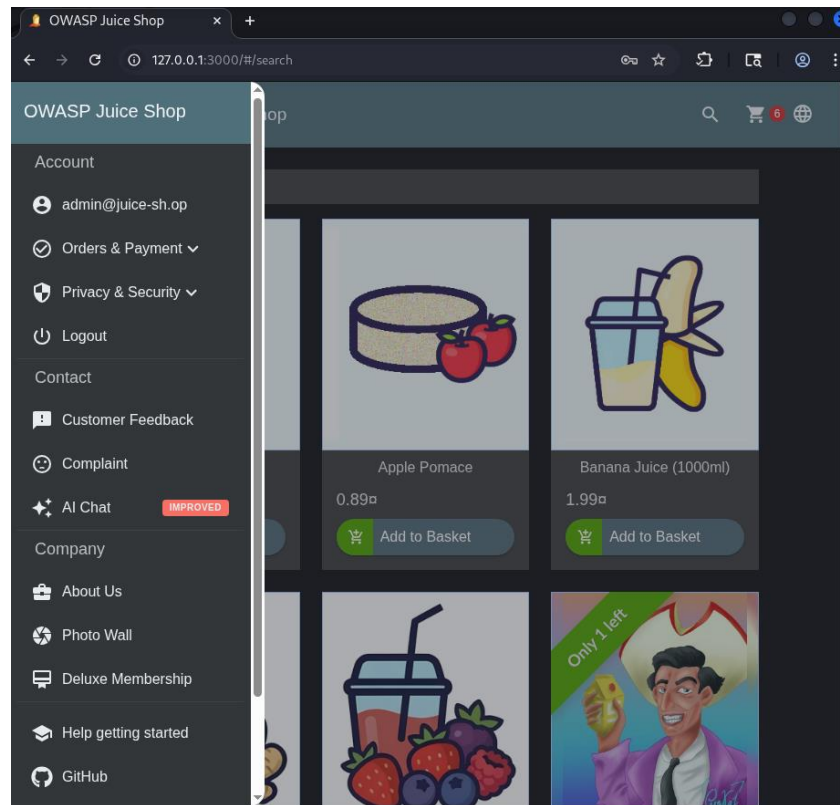


Figura 13. Sesión autenticada como administrador tras la inyección, sin contraseña válida.

Impacto: cualquier usuario no autenticado puede obtener acceso completo a la cuenta administradora sin conocer credenciales. Es la vulnerabilidad de mayor severidad identificada en esta auditoría.

Recomendación: sustituir la concatenación de cadenas por consultas parametrizadas o un ORM con bind parameters, aplicar validación estricta de entrada en el backend, y desplegar un WAF como capa de defensa adicional.

7. Conclusiones generales

La auditoría realizada sobre OWASP Juice Shop confirma el perfil de la aplicación como entorno deliberadamente vulnerable, con debilidades distribuidas en múltiples capas: exposición de rutas sensibles (/ftp), ausencia de cifrado de transporte, cabeceras de seguridad incompletas, y — el hallazgo más crítico — una vulnerabilidad de inyección SQL que permite eludir por completo la autenticación.

7.1 Resumen de hallazgos por severidad

Severidad	Hallazgo
Crítica	Inyección SQL en login con bypass de autenticación
Alta	Credenciales de administrador débiles / públicas
Alta	Ausencia total de cifrado de transporte (HTTP sin TLS)
Media	Directorio /ftp expuesto públicamente
Media	CORS configurado de forma permisiva (Access-Control-Allow-Origin: *)
Media	Cabeceras de seguridad ausentes (CSP, HSTS, Permissions-Policy, Referrer-Policy)
Baja	sitemap.xml mal configurado (sin impacto directo en seguridad)

7.2 Recomendaciones priorizadas

1. Corregir la vulnerabilidad de inyección SQL mediante consultas parametrizadas — prioridad inmediata.
2. Eliminar o forzar el cambio de credenciales administrativas por defecto.
3. Implementar TLS/HTTPS en cualquier despliegue fuera de un entorno de laboratorio aislado.
4. Restringir el acceso al directorio /ftp o eliminarlo si no es necesario en producción.
5. Configurar CORS de forma restrictiva, permitiendo únicamente los orígenes necesarios.
6. Añadir las cabeceras de seguridad ausentes: Content-Security-Policy, Strict-Transport-Security, Permissions-Policy y Referrer-Policy.

7.3 Herramientas y versiones empleadas

Herramienta	Versión
Nmap	7.99
Gobuster	3.8.2
Nikto	2.6.0
Burp Suite Community Edition	2026.2.4
Docker	28.5.2+dfsg4
OWASP Juice Shop (imagen)	bkimminich/juice-shop